

TECHNICAL IMPLEMENTATION WHITEPAPER

Jicama.tech × EventSH

Partner-Domain Storefront Embed Integration

Document version: 1.0

Issued: May 2026

Origin host: eventsh.com

Partner host: jicama.tech

Classification: Internal · Client deliverable

Jicama.tech ↔ EventSH Storefront Integration

Technical Implementation Whitepaper

Document classification: Internal / Client-deliverable **Version:** 1.0 — May 2026 **Scope:** Frontend SPA, Backend API, Edge proxy, Distribution origin **Primary target host:** `jicama.tech` (Hostinger VPS, Nginx 1.x) **Origin host:** `eventsh.com` (canonical EventSH platform)

1. Executive Summary

This document records the architecture, implementation, and operational runbook for the partner-domain storefront embed delivered between EventSH and Jicama.tech.

The integration exposes the EventSH organizer storefront for the slug `jicama-tech-singapore` at `https://jicama.tech/events`, with full event detail at `https://jicama.tech/events/<event-id>`. The visitor's URL bar remains on `jicama.tech` for the entire session — no third-party iframe is used, no redirect to `eventsh.com` occurs for browsing, and search indexers see content under the Jicama.tech apex.

Concurrently, the canonical EventSH application surface (<https://eventsh.com/>*) is preserved without behavioural change: existing organizer dashboards, public landing pages, the canonical </events> listing, the </:organizationName> storefront route, and the </:organizationName/events/:id> detail route operate exactly as they did prior to the integration. The change is strictly additive at the route table layer, gated by a hostname predicate evaluated at runtime.

The change set spans five surfaces:

1. The EventSH React SPA — partner-aware routing and link generation.
2. The EventSH Vite build configuration — absolute asset origin.
3. The EventSH NestJS backend — CORS allow-list extension.
4. The EventSH static-asset Nginx vhost — cross-origin resource policy for hashed bundle delivery.
5. The Jicama.tech Hostinger Nginx vhost — reverse-proxy intake at the </events> prefix.

2. Problem Statement

2.1 Business requirement

Jicama.tech operates an independent corporate-marketing website serving the AI / full-stack development practice. The marketing site needs to present its event programme through a first-class, branded URL space (jicama.tech/events) while the underlying ticketing, organizer management, and payment-settlement workflows continue to run on the canonical EventSH platform.

A naive solution — linking out to eventsh.com/jicama-tech-singapore — fails the requirement on two grounds:

- **Brand discontinuity.** The visitor's URL bar leaves the partner domain, which the partner considers an unacceptable degradation of their domain authority and visitor trust.
- **SEO leakage.** Search engines index event listings under the EventSH apex rather than the partner's, depriving the partner of search equity for queries containing their brand keywords.

2.2 Constraints

Constraint	Source
No modification of eventsh.com route behaviour	Operational risk control
No introduction of a new sub-domain	DNS / certificate complexity
No iframe embedding	Cross-domain cookie / UX
Asset delivery must remain on the canonical origin	Cache & cost discipline
Solution must extend to additional partners cleanly	Future-proofing
Reversible within minutes if a regression appears	Change-management policy

3. High-Level Architecture



The diagram above represents a single page-load. Subsequent in-app navigation (clicking into an event) is handled entirely client-side by React Router because the `/events*` prefix is already proxied. Refreshes on `jicama.tech/events/<event-id>` re-traverse steps 1–4 transparently.

4. Component Inventory

4.1 Partner registry — `frontend/src/lib/embedHost.ts`

A single-source-of-truth module mapping embedding hostnames to EventSH organizer slugs. Adding a new partner is a one-line append; no other file requires modification.

```
const EMBED_HOSTS: Record<string, string> = {
  "jicama.tech": "jicama-tech-singapore",
};
```

Three exports drive the integration:

Symbol	Purpose
<code>getEmbedOrgSlug()</code>	Returns the organizer slug for the current hostname, or <code>null</code> on canonical origins.
<code>getEventBasePath(orgSlug)</code>	Returns the link prefix for event-detail pages — <code>/events</code> on embed hosts, canonical path elsewhere.
<code>getCanonicalAppUrl(path)</code>	Escapes a partner domain for admin/auth flows — returns absolute <code>eventsh.com</code> URLs on embed hosts.

4.2 Route table — `frontend/src/App.tsx`

The React Router declaration is augmented with hostname-gated routes in both the unauthenticated and `user`-role authenticated blocks. The underlying matching algorithm continues to prefer static path segments over dynamic ones, so the new `/events` entry does not displace the existing `/events/:eventId/scan-tickets` route or the canonical `/:organizationName/events/:id` route on `eventsh.com`.

The host predicate is captured once into a local constant (`embedOrgSlug`) before route declaration; this guarantees that route identity is stable for a given render and avoids cascading re-renders in nested `<Routes>` blocks.

4.3 Storefront — `frontend/src/components/user/organizerStoreFront.tsx`

Two adjustments:

1. The component now accepts an optional `organizationName` prop. When present, the prop overrides `useParams().organizationName`. This lets the partner-embed route inject the slug declaratively, since the route path (`/events`) does not carry the slug as a URL segment. On canonical eventsh.com routes, the prop is unset and the existing `useParams()` resolution is preserved.
2. In-storefront navigation no longer hardcodes the canonical `/<orgSlug>/events/<id>` form. `getEventBasePath(orgSlug)` produces `/events` under a matching embed and `/<orgSlug>/events` elsewhere, keeping all event-detail links within whichever URL space the visitor entered through.

Three `<a href="/login">` references in the three storefront footer variants (full, minimal, mega) now resolve through `getCanonicalAppUrl()`. On canonical origins the helper returns the relative `/login`, preserving the client-side redirect flow. On embed origins it returns the absolute

`https://eventsh.com/login`, forcing a full-page transition off the partner domain — appropriate for admin flows which are not proxied at the Nginx layer.

4.4 Build configuration — `frontend/vite.config.ts`

Production builds emit assets with an absolute `https://eventsh.com/` base. This collapses the asset-delivery problem to a single origin regardless of how many partner domains embed the application:

```
const productionBase = env.VITE_PUBLIC_BASE || "https://eventsh.com/";
return {
  base: mode === "production" ? productionBase : "/",
  ...
};
```

Development builds retain `/` so local dev servers operate without external dependencies. The `VITE_PUBLIC_BASE` environment escape permits future migration of the bundle origin (e.g. to a dedicated CDN) without code changes.

4.5 Backend CORS allow-list — `backend/src/main.ts`

The NestJS application's origin allow-list (resolved per-request by `getAllowedDomains()`) is extended to include both the apex and the `www`-prefixed forms of the partner host:

```
return [
  "https://eventsh.com",
  "https://jicama.tech",
  "https://www.jicama.tech",
  // ...existing canonical and partner origins remain in place...
];
```

A note on the existing implementation: when an origin is not on the allow-list, the request terminates with a non-2xx response and no `Access-Control-Allow-Origin` header. The exact response code returned under rejection is an implementation detail and may surface in browser DevTools as a server error rather than a pure CORS failure — this is why pre-deployment testing of a missing-origin scenario can appear at first glance to be an unrelated backend fault.

5. Request-Flow Sequence

5.1 First page-load — `GET https://jicama.tech/events`



5.2 In-app event drill-down

When the visitor clicks an event tile, `OrganizerStorefront.handleEventClick(slug)` calls `navigate( ${getEventBasePath(orgSlug)}/${slug} )`. Under the embed, `getEventBasePath()` returns `/events`, so the resulting URL is `/events/<slug>` and React Router resolves it to `<EventFront>` without a network round-trip. The browser's history entry updates to `https://jicama.tech/events/<slug>`.

5.3 Refresh on a deep link

If the visitor reloads `https://jicama.tech/events/<event-id>` directly, the request traverses the proxy path identically to §5.1 — Jicama.tech Nginx proxies to `eventsh.com/events/<event-id>`. EventSH's catch-all SPA serving returns `index.html`, the bundle hydrates, and React Router matches `/events/:id` to `<EventFront>`. There is no special-case required for "deep-link" reloads because the proxy preserves the full path segment.

6. Edge & Origin Configuration

6.1 Jicama.tech Nginx vhost — intake block

This block must be inserted **inside** the HTTPS server block for `server_name jicama.tech;` and must precede any catch-all `location /` handler used to serve the Jicama.tech marketing site:

```
location ^~ /events {
    proxy_pass            https://eventsh.com;
    proxy_http_version    1.1;
    proxy_set_header      Host eventsh.com;
    proxy_set_header      X-Forwarded-Host jicama.tech;
    proxy_set_header      X-Forwarded-Proto https;
    proxy_set_header      X-Real-IP       $remote_addr;
    proxy_ssl_server_name  on;
    proxy_redirect         off;
    proxy_buffering        off;
}
```

Notes:

- The `^~` modifier short-circuits subsequent regex `location` matches, preventing the marketing site's PHP/static handlers from intercepting partner traffic.
- `proxy_ssl_server_name on` is required for SNI handshake with the upstream — eventsh.com presents a multi-tenant certificate selected by SNI.
- `proxy_buffering off` keeps end-to-end latency tight on SPA HTML; the payload is small and benefits more from immediate flushing than from Nginx-side coalescing.

Activation:

```
sudo nginx -t && sudo systemctl reload nginx
```

6.2 EventSH Nginx vhost — asset cross-origin block

The Vite build emits ES-module script tags and `crossorigin`-flagged CSS link tags. Both require an explicit `Access-Control-Allow-Origin` header on cross-origin fetches; without it, the browser's fetch fails with a `net::ERR_FAILED` and the bundle does not execute.

Inside the HTTPS server block for `server_name eventsh.com;`:

```
location /assets/ {
    add_header Access-Control-Allow-Origin "https://jicama.tech" always;
    add_header Access-Control-Allow-Methods "GET, OPTIONS" always;
    add_header Access-Control-Max-Age 86400 always;
    add_header Cache-Control "public, max-age=31536000, immutable" always;

    if ($request_method = OPTIONS) {
        return 204;
    }

    try_files $uri =404;
}
```

For two or more partners, the allow-list collapses cleanly into an origin-keyed map:

```
map $http_origin $cors_origin {
  default                "";
  "https://jicama.tech"   $http_origin;
  "https://www.jicama.tech" $http_origin;
  "https://anotherpartner.com" $http_origin;
}
```

with `add_header Access-Control-Allow-Origin $cors_origin always;` inside the `/assets/` block.

6.3 EventSH backend — application-layer CORS

The Nest CORS middleware reads `Origin` on every API request and admits or rejects against the allow-list resolved by `getAllowedDomains()`. Deployment of the updated `main.ts` is performed by the standard build-and-restart cycle:

```
cd backend
npm run build
# Restart the API service via your standard process manager.
```

7. Verification Procedure

7.1 Regression check — `eventsh.com`

Each of the following must behave **identically** to the pre-integration baseline. Any deviation constitutes a regression and is grounds for immediate rollback.

URL	Expected outcome
<code>https://eventsh.com/</code>	Landing page renders.
<code>https://eventsh.com/events</code>	Canonical events listing renders.
<code>https://eventsh.com/jicama-tech-singapore</code>	Storefront renders.
<code>https://eventsh.com/jicama-tech-singapore/events/<event-id></code>	Event detail renders.
<code>https://eventsh.com/organizer/login</code>	Organizer login page renders.
<code>https://eventsh.com/organizer-dashboard</code> (authenticated)	Organizer dashboard renders.
Other tenant storefront flows	Behave exactly as before this integration.

7.2 Integration check — `jicama.tech`

URL	Expected outcome
<code>https://jicama.tech/</code>	Existing Jicama.tech marketing site renders unchanged.
<code>https://jicama.tech/events</code>	Jicama-Tech-Singapore storefront renders; URL bar stays.
<code>https://jicama.tech/events/<event-id></code>	Event detail renders; URL bar stays.
<code>https://jicama.tech/events/<event-id></code> (reload)	Identical render; no 404.
Footer "Organizer Login" click	Full navigation to <code>https://eventsh.com/organizer/login</code> .

7.3 Network-layer assertions (DevTools → Network)

- Asset requests show `Status: 200`, `Initiator: Other`, `Response Headers: access-control-allow-origin: https://jicama.tech`.
- API requests under `https://eventsh.com/api/*` show `Status: 200` with the same CORS header.
- No requests visible under `https://jicama.tech/assets/` or `https://jicama.tech/api/` — both must terminate on `eventsh.com`.

8. Operational Considerations

8.1 Failure modes & remediation

Symptom	Root cause	Remediation
<code>jicama.tech/events</code> → Jicama.tech 404	Nginx location block missing or shadowed	Re-add block, place before catch-all, <code>nginx -t</code> .
Blank page on <code>jicama.tech/events</code>	Asset CORS rejected	Add <code>/assets/</code> allow-origin block on eventsh.com Nginx.
<code>net::ERR_FAILED 500</code> on <code>/api/*</code> calls	Backend allow-list omits partner	Append origin to <code>getAllowedDomains()</code> , redeploy backend.
"No routes matched location '/organizer/login'"	Admin path not proxied; React Router has no embed-side route	Confirmed expected; use <code>getCanonicalAppUrl()</code> helper.
Refresh on <code>/events/<id></code> returns Jicama.tech 404	<code>^~</code> modifier missing or block scoped wrong	Verify location <code>^~ /events</code> is at server-block scope.

8.2 Rollback

Two rollback paths are available, ordered by least disruption:

1. **Edge-only rollback** — comment out the `location ^~ /events { ... }` block on Jicama.tech Nginx, reload. Partner traffic at `/events` reverts to Jicama.tech's prior behaviour (404 or marketing-page handler). EventSH operates entirely unaffected; no code rebuild required. Time to revert: < 30 seconds.
2. **Application rollback** — `git revert` the integration commits on the EventSH frontend, rebuild, redeploy. `getEmbedOrgSlug()` will return `null` on all hosts, the gated routes will not register, and the storefront component will revert to `useParams()` -only slug resolution.

8.3 Adding a new partner

A new partner integration requires three actions and no architectural review:

1. Append the partner's hostname → organizer-slug mapping to `EMBED_HOSTS` in `frontend/src/lib/embedHost.ts`. Rebuild and redeploy the EventSH frontend.
2. Extend the `Access-Control-Allow-Origin` allow-list on the eventsh.com `/assets/` Nginx block (preferably via the `$cors_origin` map shown in §6.2) and on `getAllowedDomains()` in the EventSH backend. Reload Nginx and redeploy the backend.
3. On the partner's Nginx, drop in the same `location ^~ /events { ... }` block from §6.1.

The integration is operational once all three are deployed.

8.4 SEO / canonicalization

The same HTML is served under two URLs (`jicama.tech/events` and `eventsh.com/jicama-tech-singapore`). Search engines treat this as duplicate content unless instructed otherwise. The recommended follow-up is to emit a `<link rel="canonical">` header inside the storefront `<Helmet>` block whose `href` resolves dynamically to the canonical eventsh.com URL when `getEmbedOrgSlug()` returns non-null. This preserves the partner's URL while crediting eventsh.com as the authoritative source for indexing purposes.

9. File-Level Change Index

File	Change type	Purpose
<code>frontend/src/lib/embedHost.ts</code>	New	Partner registry, base-path helper, canonical escape.
<code>frontend/src/App.tsx</code>	Modified	Gated <code>/events</code> & <code>/events/:id</code> routes.
<code>frontend/src/components/user/organizerStoreFront.tsx</code>	Modified	Prop-overridable slug, host-aware navigation, canonical login link.
<code>frontend/vite.config.ts</code>	Modified	Production asset base set to <code>https://eventsh.com/</code> .
<code>backend/src/main.ts</code>	Modified	CORS allow-list extension.
<code>frontend/JICAMA-TECH-EMBED-GUIDE.md</code>	New	Engineering runbook.
<code>JICAMA-TECH-INTEGRATION-WHITEPAPER.md</code>	New	This document.

Nginx vhost changes (server-side, not version-controlled in this repo):

Host	File path (typical)	Purpose
<code>jicama.tech</code>	<code>/etc/nginx/sites-available/jicama.tech</code>	<code>/events</code> reverse-proxy.
<code>eventsh.com</code>	<code>/etc/nginx/sites-available/eventsh.com</code>	<code>/assets/</code> cross-origin headers.

10. Glossary

- **Embed host** — A partner domain whose Nginx layer reverse-proxies a defined path prefix into the EventSH application surface.
- **Canonical origin** — `eventsh.com`. The authoritative source of EventSH HTML, JS, CSS, and API responses.
- **Slug** — A URL-safe identifier for an organizer's storefront. Stored on the `OrganizerStore` document.
- **EMV TLV** — Europay-Mastercard-Visa Tag-Length-Value encoding, used by PayNow and UPI to express payment metadata in QR codes. (Referenced here for cross-cutting context with the parallel payment-QR work; not strictly part of the embed integration.)
- **Embed slug** — The organizer slug returned by `getEmbedOrgSlug()` for the current `window.location.hostname`.

11. Sign-Off

Role	Verification responsibility
Backend engineer	CORS allow-list, build pipeline, restart-cycle health.
Frontend engineer	Vite base flag, route table assertions, link audits.
DevOps / SRE	Both Nginx vhosts; certificate validity; reload procedure.
Product / Partner mgmt	Live URL walk-through against §7.1 and §7.2.

The integration is considered delivered when each row above has been signed off against the verification matrices in §7.

End of document.